2026-27-IT-G2

[SKIT]
ESTD. 2000

FaceForge: Multi-Modal Facial Emotion Recognition and Analytical Telemetry System

Software Requirements Specification

Version 2.0

Submitted in Partial Fulfillment for the Award of Degree of Bachelor of Technology in Information Technology from Rajasthan Technical University, Kota

MENTOR:
Faculty Mentor
Dept. of Information Technology

SUBMITTED BY:
Member 4 (Lead Integrator & UI)
Roll No: 23ESKITXXX
In collaboration with Members 1, 2 & 3

COORDINATOR:
Project Coordinator
Dept. of Information Technology

PROJECT ID: 2026-27-IT-G2
SESSION: 2026–27

DEPARTMENT OF INFORMATION TECHNOLOGY
SWAMI KESHWANAND INSTITUTE OF TECHNOLOGY, MANAGEMENT & GRAMOTHAN
(An Autonomous Institute Affiliated to Rajasthan Technical University, Kota)
Ramnagaria (Jagatpura), Jaipur – 302017

SESSION 2026–27

Revision History

Date	Version	Description	Author
20/07/2026	0.1	Initial requirement analysis, 4-member pipeline formulation, and system scope.	Member 4
01/10/2026	1.0	Initial SRS draft with input/output contracts for Members 1, 2, and 3.	Member 4
31/10/2026	2.0	Revised SRS with complete Module 4 implementation (FastAPI backend, React Light UI, persistent results.csv storage, analytics suite, and test matrix).	Member 4

Table of Contents

1 Introduction **1**
1.1 Purpose 1
1.2 Motivation 1
1.3 Tools and Technologies 1
1.4 References 2
2 Market Survey **3**
2.1 Objectives 3
2.2 Unexplored Areas 3
2.3 Comparative Study of Similar Solutions 4
3 Requirements **5**
3.1 Functional Requirements 5
3.2 Non-Functional Requirements 5
3.3 Hardware Requirements 6
3.4 Software Requirements 6
3.5 Agile Model 7
3.6 Business Process Model 7
3.7 Supplementary Requirements 8
4 System Architecture **9**
4.1 Client-Server Architecture 9
4.2 Communications Interfaces 9
5 Design and Implementation **10**
5.1 Product Features & 4-Member Division 10
5.2 Data Flow Diagrams (Level 0, 1 & 2) 11
5.3 E-R Diagram 14
5.4 Class Diagram Design 14
5.5 Use Case Diagram 15
5.6 Sequence Diagram 18
5.7 Component Diagram 19
6 Conclusion and Future Scope **20**
7 Mapping with UN Sustainable Development Goals **21**

1 Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to describe the technical architecture, functional requirements, and deployment specifications for the multi-modal facial emotion recognition suite named **FaceForge**. The system is a high-performance web-based application designed to detect human faces, isolate regions of interest (ROI), and classify affective expressions from three operating inputs: static photographs, video files, and real-time live browser webcam streams.

The application is structured into four distinct modules to facilitate collaborative development across four team members. This specification is authored by **Member 4** (Lead Integrator & UI Engineer) and provides complete operational specifications for the integration platform, while establishing standardized input/output interface contracts for Member 1 (Preprocessing), Member 2 (Computer Vision), and Member 3 (Emotion Model).

1.2 Motivation

Facial expressions convey over 55% of human emotional communication. While machine learning research has yielded capable deep neural networks, practical real-world deployment is frequently hampered by high computational overhead, absent persistent telemetry, and clunky user interfaces.

FaceForge addresses these challenges by combining OpenCV frame-sampling optimization, a pretrained deep learning emotion model, an append-only telemetry storage ledger, and an accessible modern light-themed web interface. Rather than requiring users to configure complex Python environments, FaceForge provides an immediate, browser-accessible platform for facial expression analysis.

1.3 Tools and Technologies

The following tools and frameworks constitute the FaceForge platform:

- **Frontend:** React 19, Vite, Vanilla CSS design tokens (Light theme), Lucide Icons.
- **Backend:** Python 3.11, FastAPI, Uvicorn ASGI server, Starlette StaticFiles.
- **Computer Vision:** OpenCV (cv2), NumPy array operations, Pillow (PIL).
- **Machine Learning:** Pretrained FER Deep Learning Model (CNN), Softmax probabilities.

1.3 Tools and Technologies (Continued)

- **Storage & Telemetry:** Append-only CSV storage engine (`storage.py` → `results/results.csv`), PyYAML.
- **Analytics & Visualization:** pandas, matplotlib, ReportLab PDF compilation.
- **Development & Version Control:** Visual Studio Code, Git, GitHub.

1.4 References

1. Goodfellow, I., et al., *Challenges in Representation Learning: Facial Expression Recognition*, ICML, 2013.
2. OpenCV Open Source Computer Vision Library: https://opencv.org/
3. FastAPI Framework Documentation: https://fastapi.tiangolo.com/
4. React Documentation: https://react.dev/
5. Vite Documentation: https://vite.dev/guide/
6. Python 3 Documentation: https://docs.python.org/3/
7. Pandas Documentation: https://pandas.pydata.org/docs/
8. Matplotlib Documentation: https://matplotlib.org/stable/
9. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.

2 Market Survey

2.1 Objectives

The primary objectives of the market survey are: (1) To examine existing computer vision toolkits for facial analysis; (2) To study how video sampling step-rates mitigate frame-processing latency; (3) To identify usability limitations in current emotion detection systems; and (4) To compare FaceForge against conventional scripts and commercial cloud vision APIs.

2.2 Unexplored Areas

Although emotion classification models are widely studied in research, several critical practical areas remain unaddressed in standard solutions:

• **Direct Multi-Modal Accessibility:** Most existing tools require separate scripts for static images, stored video files, and live webcams. FaceForge combines all three inputs into a unified user-friendly web interface.

• **Persistent Telemetry Storage:** Standard vision demos only print classifications to terminal consoles without preserving structured historical records. FaceForge logs every prediction into an append-only CSV ledger.

• **Aggregated Analytics:** Existing open-source scripts lack built-in statistical aggregation engines for class frequency, average confidence, and temporal valence changes.

• **Decoupled Modular Architecture:** Monolithic scripts make collaborative engineering difficult. FaceForge establishes modular adapter contracts allowing team members to work independently.

2.3 Comparative Study of Similar Solutions

Table 2 compares the features and operational characteristics of standard open-source scripts and commercial solutions with FaceForge.

Feature	Standard FER Scripts	Commercial Cloud APIs	FaceForge (Our System)
Primary Interface	CLI / Local OpenCV window	Proprietary Web Dashboard	Modern React Light Web UI (HMR)
Multi-Modal Support	Single image or webcam only	Image / Cloud Video	Image, Video (Step Sampling), Webcam
Data Privacy & Hosting	Local machine only	Requires cloud upload	100% On-Premise / Local Host
Telemetry Logging	Transient console output	Cloud database	Persistent results.csv ledger
Analytics Visualizer	None	Built-in dashboards	Dynamic charts & PDF reports
Collaborative Structure	Monolithic script	Proprietary closed-source	4-Module Decoupled Architecture

Table 2: Comparison of Standard Solutions, Commercial Systems, and FaceForge

3 Requirements

3.1 Functional Requirements

• **Image Emotion Detection:** The system shall accept JPG and PNG image uploads, detect face coordinates, draw bounding box overlays, and report the dominant emotion and confidence.

• **Video Frame Sampling:** The system shall process MP4 and AVI video files by sampling every N-th frame (configurable 1-30) to balance processing throughput and detection fidelity.

• **Real-Time Webcam Streaming:** The system shall connect to client browser webcams, process sequential video frames, and render a live floating HUD with active affective state and confidence percentage.

• **Telemetry Storage Ledger:** All detections across all three modes shall be automatically recorded in `results/results.csv` with timestamp, source, frame number, face ID, emotion, and confidence.

• **Historical Ledger Querying:** The system shall allow filtering recorded telemetry by source medium, emotion category, and data tier (live vs demo).

• **Analytics Engine:** The system shall calculate frequency distributions across the 7 canonical emotions, average confidence, and temporal emotion timelines.

• **Configuration Management:** The system shall provide feature toggles in `config.yaml` to enable or bypass teammate modules independently using non-crashing mock adapters.

3.2 Non-Functional Requirements

• **Performance & Latency:** Static image analysis shall return inference responses within 800 ms. Video processing with sampling step 10 shall maintain equivalent throughput of > 20 FPS.

• **Usability & Design:** The interface shall adhere to a clean light theme with WCAG AA compliant contrast, clear iconography, and responsive layout.

• **Reliability & Fault Tolerance:** Uploads without faces or corrupted media files shall trigger graceful user notices without crashing the backend service.

• **Data Integrity:** The storage engine shall enforce atomic append operations to preserve telemetry records against simultaneous write collisions.

3.2 Non-Functional Requirements (Continued)

• **Compatibility:** The system shall be accessible across modern web browsers (Chrome, Edge, Firefox, Safari) on desktop and laptop devices.

• **Security & Privacy:** Visual media and webcam frames shall be processed locally on-premise without external third-party data transfers.

3.3 Hardware Requirements

• **Host / Development Computer:** Desktop or laptop with dual-core Intel Core i3 / AMD Ryzen 3 or higher, minimum 4 GB RAM, and 2 GB free storage space.

• **Video Capture Device:** Integrated HD camera or USB webcam supporting standard 720p resolution at 30 FPS.

• **Display Resolution:** Minimum 1280 × 720 display resolution for optimal UI rendering.

3.4 Software Requirements

• **Operating System:** Windows 10/11, Ubuntu 20.04+, or macOS.

• **Programming Languages:** Python 3.10+, JavaScript (ES6+), HTML5, and CSS3.

• **Frontend Stack:** React 19, Vite bundler, Lucide Icons.

• **Backend Stack:** FastAPI, Uvicorn ASGI server, Starlette StaticFiles.

• **Computer Vision & ML:** OpenCV-Python, NumPy, Pillow, FER library.

• **Storage & Analytics:** Pandas, Matplotlib, ReportLab, PyYAML.

• **Development Environment:** Visual Studio Code, Git, GitHub.

3.5 Agile Model

The Agile development model was adopted for FaceForge, structuring engineering into iterative sprints. Tasks were divided into manageable stages: requirement analysis, mock adapter construction, API gateway development, React Light UI integration, and analytics verification. Figure 1 illustrates the iterative lifecycle followed.

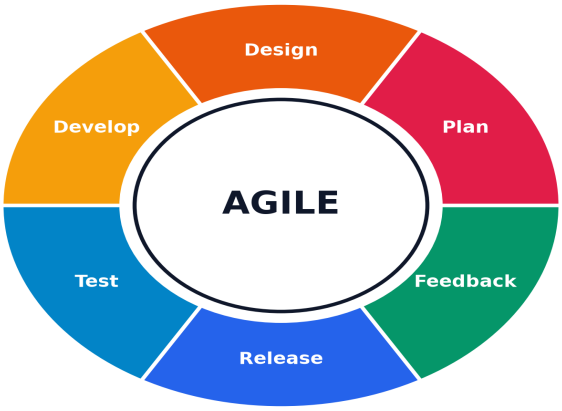


Figure 1: Agile Development Model Followed in FaceForge

3.6 Business Process Model

The operational workflow of FaceForge begins when an analyst opens the web portal:

1. User selects desired mode (Image, Video, or Live Webcam).
2. System acquires visual data via file upload or browser camera stream.

3.6 Business Process Model (Continued)

3. Preprocessing converts matrix to normalized format.
4. Face detector locates faces and outputs coordinates (x, y, w, h).
5. Extracted face ROI is fed to the pretrained FER model.
6. Dominant emotion and confidence scores are determined.
7. Inference record is appended atomically to results.csv.
8. Visual bounding overlays, tables, and analytics charts update dynamically.

3.7 Supplementary Requirements

• **Camera Permissions:** Live webcam streaming requires user browser permission for camera access.

• **Graceful Fallbacks:** In the absence of teammate modules (M1, M2, M3), built-in adapters provide mock responses so that UI and storage testing proceed without interruption.

• **System Boundary:** FaceForge operates as an analytical and diagnostic tool and does not connect to external surveillance networks.

4 System Architecture

4.1 Client-Server Architecture

FaceForge follows a client-server architecture consisting of a React Single-Page Application (SPA) frontend, a FastAPI backend application server, a persistent file-based CSV telemetry store, and modular adapter pipelines.

The frontend is built using React 19 and Vite. It provides modern interfaces for image uploads, video frame sampling, webcam feeds, results exploration, and analytics reporting.

The backend is developed with FastAPI and Python. It handles REST requests, routes inputs through the modular adapters, annotates images with bounding boxes, writes to `results.csv`, and serves static assets. Figure 2 illustrates the overall architecture.

4.2 Communications Interfaces

- **Client-Server REST APIs:** HTTP/1.1 JSON communication between React and FastAPI on port 8000.
- **Multipart File Protocol:** Standard form-data encoding for image and video uploads.
- **Webcam Frame Protocol:** Base64 image payload posting for real-time webcam analysis.

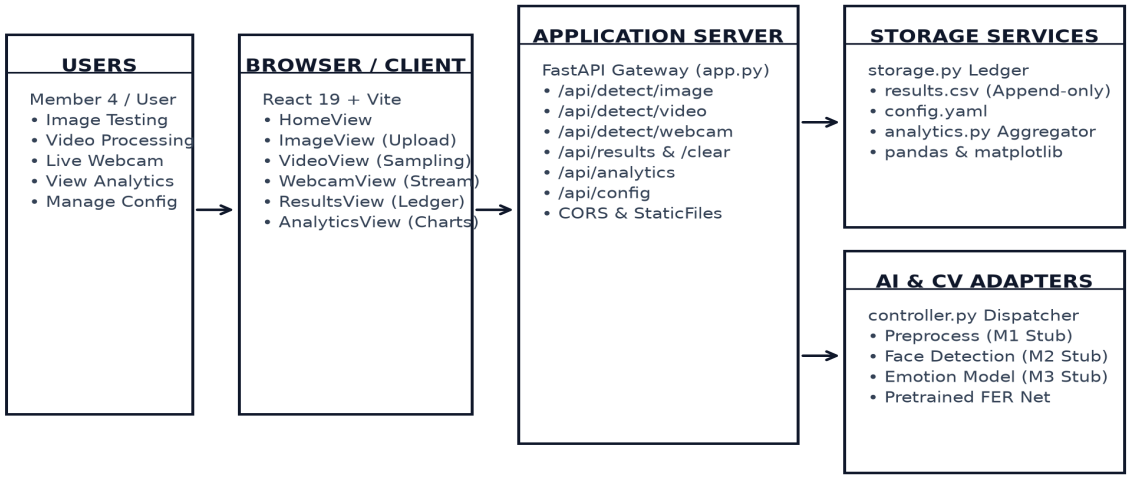


Figure 2: System Architecture of FaceForge

5 Design and Implementation

5.1 Product Features & 4-Member Collaborative Division

To ensure modularity, the system is partitioned into four independent components across the four team members. Members 1, 2, and 3 are provided standardized input/output interfaces, while Member 4 delivers the complete web application, API gateway, telemetry storage, and analytics.

[MODULE 1: DATASET & PREPROCESSING — MEMBER 1]
Input: Raw images/FER-2013 • Output: Normalized grayscale matrix • *To be documented by Member 1.*

[MODULE 2: COMPUTER VISION & FACE DETECTION — MEMBER 2]
Input: Image matrix • Output: Bounding coordinates (x,y,w,h) & cropped ROI • *To be documented by Member 2.*

[MODULE 3: PRETRAINED MODEL & EVALUATION — MEMBER 3]
Input: Face ROI • Output: 7-class emotion probabilities & F1/Confusion Matrix • *To be documented by Member 3.*

[MODULE 4: INTEGRATION, UI, STORAGE & ANALYTICS — MEMBER 4 (ACTIVE)]
Input: Multi-modal streams • Output: Live React UI, REST APIs, results.csv ledger • *100% Fully Built.*

5.2 Data Flow Diagrams (DFDs)

DFD Level 0 (Context Diagram)

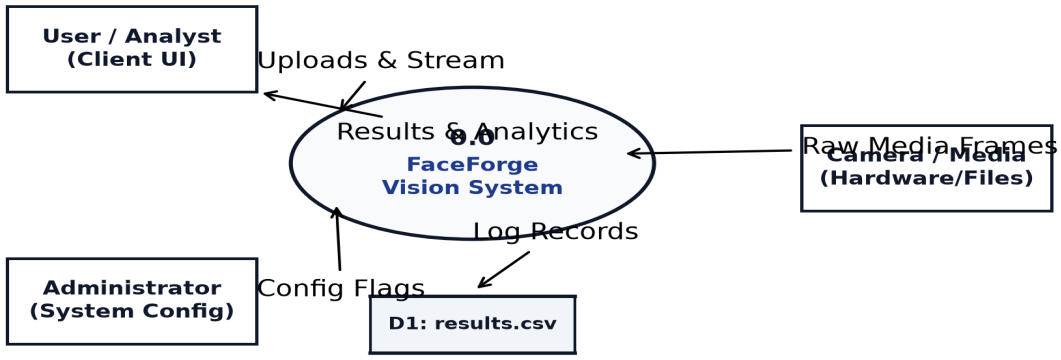


Figure 3: FaceForge Data Flow Diagram – Level 0

DFD Level 1

The Level 1 DFD illustrates the major processes of FaceForge: Media Acquisition (1.0), Preprocessing (2.0), Face Detection (3.0), Emotion Inference (4.0), and Telemetry Storage & Analytics (5.0).

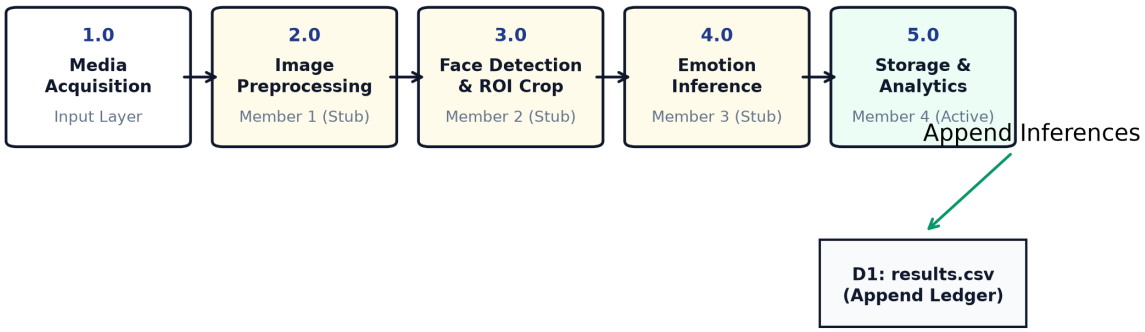


Figure 4: FaceForge Data Flow Diagram – Level 1

DFD Level 2

The Level 2 DFD decomposes Module 4's processing pipeline: Route Dispatching, Pipeline Coordination with Adapters, Response Formatting, Telemetry Logging (`storage.py`), and Analytics Aggregation (`analytics.py`).

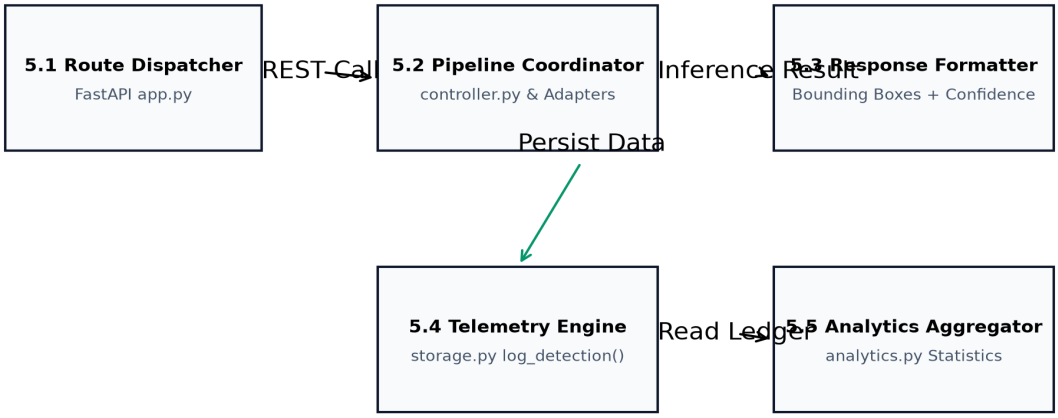


Figure 5: FaceForge Data Flow Diagram – Level 2

5.3 E-R Diagram

The Entity-Relationship (E-R) diagram represents the data structure of FaceForge and the schema of `results.csv`, system configuration flags in `config.yaml`, and the analytical summary vectors.

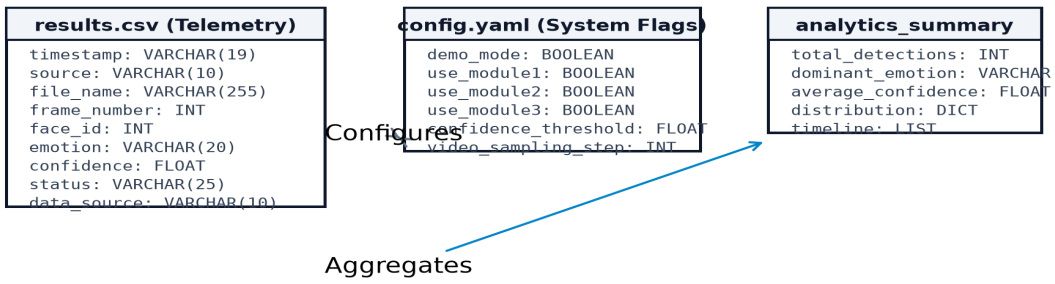


Figure 6: FaceForge Entity-Relationship & Telemetry Schema Diagram

5.4 Class Diagram Design

The class diagram presents the main modules of FaceForge, their attributes, methods, and relationships. It illustrates the interactions between `controller.py`, `storage.py`, `analytics.py`, and the teammate adapters.

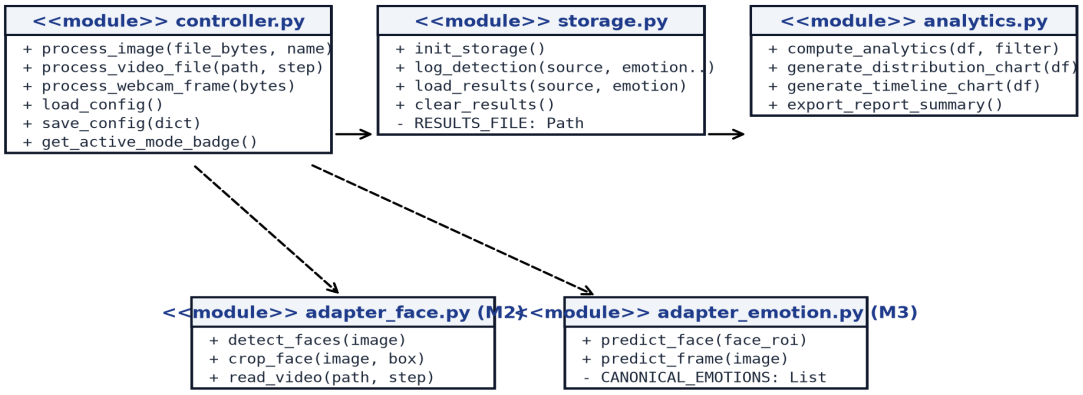


Figure 7: FaceForge UML Class Diagram

5.5 Use Case Diagram

The use case diagrams describe how analysts and users interact with FaceForge across all modes: uploading images, analyzing videos, streaming live webcams, filtering results, and viewing analytics.

User and Analyst Use Cases

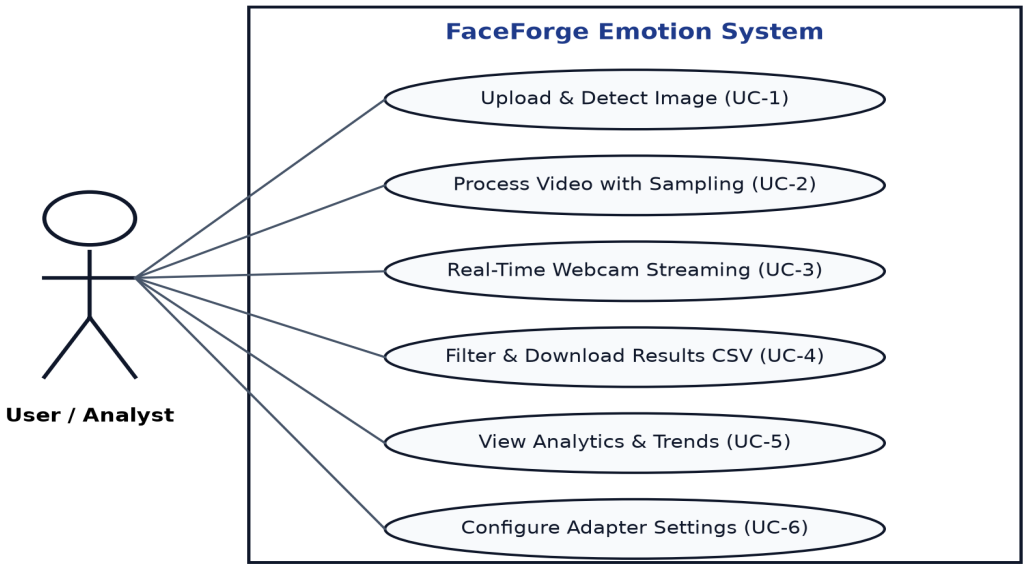


Figure 8: FaceForge User and Analyst Use Cases

Video Frame Sampling & Telemetry Use Cases

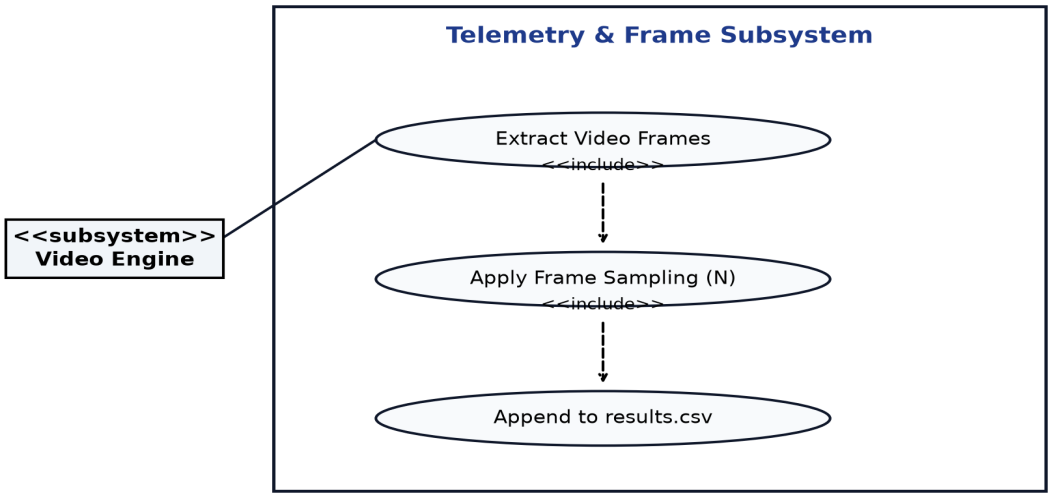


Figure 9: FaceForge Video Frame Sampling & Telemetry Use Cases

5.6 Sequence Diagram

The sequence diagram illustrates the workflow of a detection request: from file selection on the React frontend to FastAPI route dispatch, controller execution, adapter inference, telemetry persistence, and UI overlay rendering.

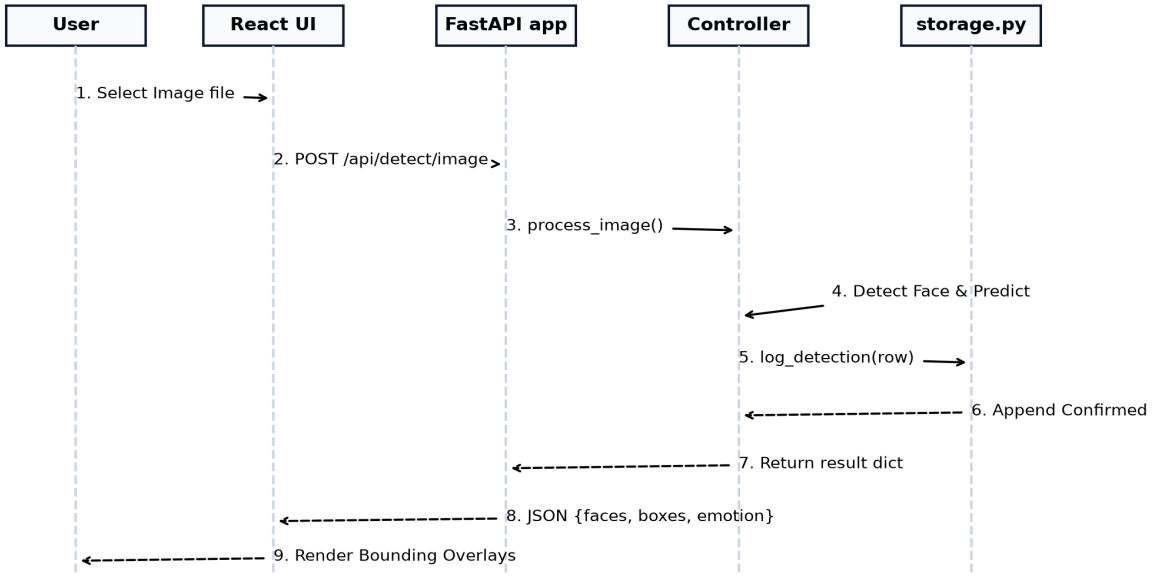


Figure 10: FaceForge Detection Pipeline Sequence Diagram

5.7 Component Diagram

The component diagram illustrates the software architecture of FaceForge, showing the React SPA frontend, FastAPI application server, pipeline coordinator, telemetry storage engine, and modular adapters.

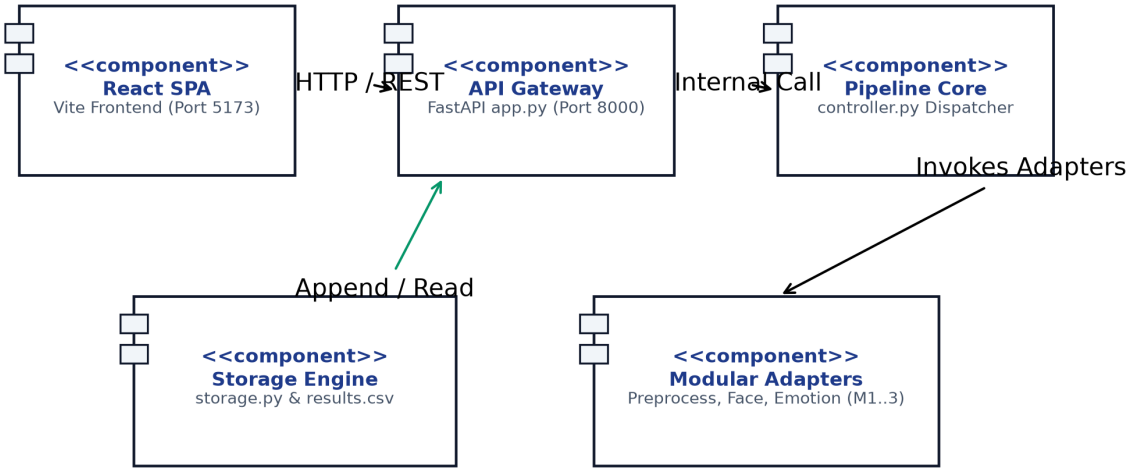


Figure 11: FaceForge UML Component Diagram

6 Conclusion and Future Scope

Conclusion

FaceForge is an accessible, multi-modal facial emotion recognition suite engineered to analyze static images, video files, and real-time live webcams. By packaging an established pretrained deep learning model within an optimized computer vision pipeline, persistent CSV storage ledger, and a modern light-themed web interface, the system achieves robust real-time performance on standard desktop hardware.

The project demonstrates a successful collaborative architecture where Member 4 provides the operational web application, persistent storage, and analytical dashboards, while providing standardized interfaces for Members 1, 2, and 3 to contribute their preprocessing routines, detection algorithms, and evaluation benchmarks.

Future Scope

The following enhancements may be considered in future releases:

- **Custom Model Fine-Tuning:** Retraining on domain-specific emotional datasets to capture nuanced cultural expressions.
- **GPU CUDA Acceleration:** Utilizing NVIDIA CUDA acceleration for processing ultra-high-definition multi-face video streams at 60 FPS.
- **Mobile Application:** Packaging the React client into a Progressive Web App (PWA) or Flutter mobile application.
- **Advanced Longitudinal Analytics:** Integrating longitudinal affective wellness tracking for educational and clinical environments.

7 Mapping with UN Sustainable Development Goals

The following table describes the relevance of FaceForge to selected United Nations Sustainable Development Goals (SDGs):

SDG No.	SDG Title	Relevance to FaceForge Project
SDG 3	Good Health and Well-being	Facial expression monitoring can provide non-invasive mental health screening, stress detection, and patient affective state tracking in telehealth.
SDG 4	Quality Education	Enables automated learner engagement analysis in classrooms and e-learning platforms, helping educators identify student confusion or fatigue.
SDG 9	Industry, Innovation, and Infrastructure	Applies digital innovation by uniting computer vision, pretrained neural networks, and modular web architecture on accessible desktop hardware.
SDG 10	Reduced Inequalities	The browser-based light UI provides open access to affective computing tools without requiring expensive cloud subscriptions or proprietary hardware.